

System Architecture & Design Document:

Accessible Multi-Factor Authentication (A-MFA) for Visually Impaired Users

Version: 1.0.0 | Status: Approved Architecture Specification | Classification: Engineering Design

Index number	Name	Contribution
230153C	Disanayaka K.T.K.	Frontend Implementation, Screen Reader Accessibility Specifications
230361L	Lakmal H.M.P.	Authentication Architecture, Technical Architecture and Workflow
230642B	Thennakoon T.M.S.S.	Summary & problem definition Functional requirements
230702K	Wickramaarachchi W.A.A.U.	Database schema and data models

Table of Content

System Architecture & Design Document.....	1
Accessible Multi-Factor Authentication (A-MFA) for Visually Impaired Users.....	1
Table of Content.....	2
1. Executive Summary & Problem Definition.....	3
2. System Goals, Requirements.....	3
2.1 Functional Requirements.....	3
2.2 Non-Functional Requirements.....	4
3. Authentication Factor Architecture & Comparison Matrix.....	4
3.1 Authentication Factors.....	4
3.2 Authentication Factor Comparison Matrix.....	5
3.3 Selected Authentication Strategy.....	6
3.4 Authentication Decision and Fallback Strategy.....	7
4. Detailed Technical Architecture & Workflows.....	7
4.1 Overall System Architecture.....	7
4.2 Registration and Authentication Workflow.....	8
4.3 WebAuthn / FIDO2 Authentication Workflow.....	9
4.4 Accessible Fallback Authentication.....	9
4.5 Technology Stack.....	10
5. Frontend Implementation & Screen Reader Accessibility Specifications.....	11
5.1 Single Consolidated Input & Live Region Handling.....	11
5.2 Client-Side JavaScript Assertion Script.....	13
6. Database Schema & Data Models.....	17
7. Threat Modeling & Security Controls.....	18
8. Quality Assurance & Assistive Technology Testing Matrix.....	19
9. Architectural Conclusion & Roadmap.....	20
9.1 Future Improvements.....	20
9.2 Design Limitations.....	20
10. Security and Design Considerations.....	21
10.1 Credential Protection.....	21
10.2 Challenge and Replay Protection.....	21
10.3 Session Security.....	21
10.4 Audit Logging.....	21
10.5 Accessibility and Security Balance.....	21
11. Design Decision Summary.....	22
12. Conclusion.....	23
13. References.....	23

1. Executive Summary & Problem Definition

Traditional Multi-Factor Authentication (MFA) architectures are not focused on visually impaired people. Most MFA systems are based on short-lived numeric Time-Based One-Time Passwords (TOTP) from external authenticator apps, visual CAPTCHAs, or interpreting unlabelled dynamic UI components under a small time constraint. For users relying on screen reading software (such as NVDA, JAWS, VoiceOver, and TalkBack) or refreshable Braille displays, these mechanisms introduce cognitive friction, latency, and high error rates, frequently resulting in account lockouts and security workarounds.

The Accessible Multi-Factor Authentication (A-MFA) system re-engineers the secondary verification paradigm by prioritizing cryptographic, phishing-resistant, touch-activated protocols (FIDO2 / WebAuthn) and accessible out-of-band channels. This document establishes the end-to-end technical specifications, threat models, database schemas, frontend WAI-ARIA compliance guidelines, and backend service contracts necessary to deploy an enterprise-grade MFA solution.

2. System Goals, Requirements

2.1 Functional Requirements

- **Primary Second Factor (Hardware/Biometric WebAuthn):** Enable zero-transcription cryptographic authentication through physical hardware tokens (e.g., capacitive touch security keys via USB or NFC) and platform biometrics (Touch ID, Face ID, Windows Hello).
- **Secondary Push Factor:** Provide a mobile push notification mechanism offering high-contrast, single-action authorization buttons integrated with native OS accessibility hooks.
- **Accessible Out-of-Band Fallback (IVR Call):** Offer an automated Interactive Voice Response (IVR) phone service delivering spaced dual-tone multi-frequency (DTMF) keypad validation ("Press 1 to authorize login") rather than dictation of high-stress OTP strings.
- **Audio OTP Delivery (Regulated Alternative):** When numeric OTP is strictly necessary, provide high-definition, phonetic, human-spaced speech generation with replay controls and keyboard shortcuts.
- **Dynamic Session Timing Extension:** Provide native WCAG 2.2 Guideline 2.2 compliant session timeout warnings with single-keystroke extension requests (+5 minutes).

2.2 Non-Functional Requirements

Security: The system shall protect authentication credentials, challenges, sessions and audit information against unauthorized access.

Accessibility: Authentication workflows shall support visually impaired users through screen readers, keyboard navigation, audio feedback and accessible controls.

Usability: Authentication steps shall minimize manual transcription and unnecessary visual interaction.

Performance: Authentication requests and verification processes should respond within an acceptable time without causing unnecessary delays for users.

Availability: The system should provide alternative authentication mechanisms when the preferred authentication method is unavailable.

Privacy: Personal information, authentication data and audit records shall only be accessible to authorized components and users.

Maintainability: The architecture should allow authentication mechanisms and accessibility features to be modified or extended without redesigning the complete system.

Scalability: The authentication service and database architecture should support increasing numbers of users and authentication requests.

3. Authentication Factor Architecture & Comparison Matrix

The proposed Accessible Multi-Factor Authentication (A-MFA) system combines strong authentication mechanisms with accessibility-focused interaction methods. The system prioritizes phishing-resistant authentication while providing alternative verification mechanisms for users who may be unable to use a particular authentication method.

3.1 Authentication Factors

The system considers the following authentication mechanisms:

- **Password/PIN:** A knowledge-based authentication factor requiring the user to provide a secret credential.
- **FIDO2/WebAuthn:** A possession-based, cryptographic authentication mechanism using a registered security key or platform authenticator.
- **Platform Passkeys/Biometrics:** Platform-based authentication using a passkey protected by a device PIN, fingerprint, or facial recognition where supported.
- **Mobile Push Authentication:** A possession-based mechanism where the user approves an authentication request from a registered mobile device.
- **IVR Authentication:** An accessible telephone-based fallback mechanism using voice instructions and keypad input.
- **Audio OTP:** An accessible fallback mechanism in which a time-limited one-time password is delivered through audio and entered by the user.

3.2 Authentication Factor Comparison Matrix

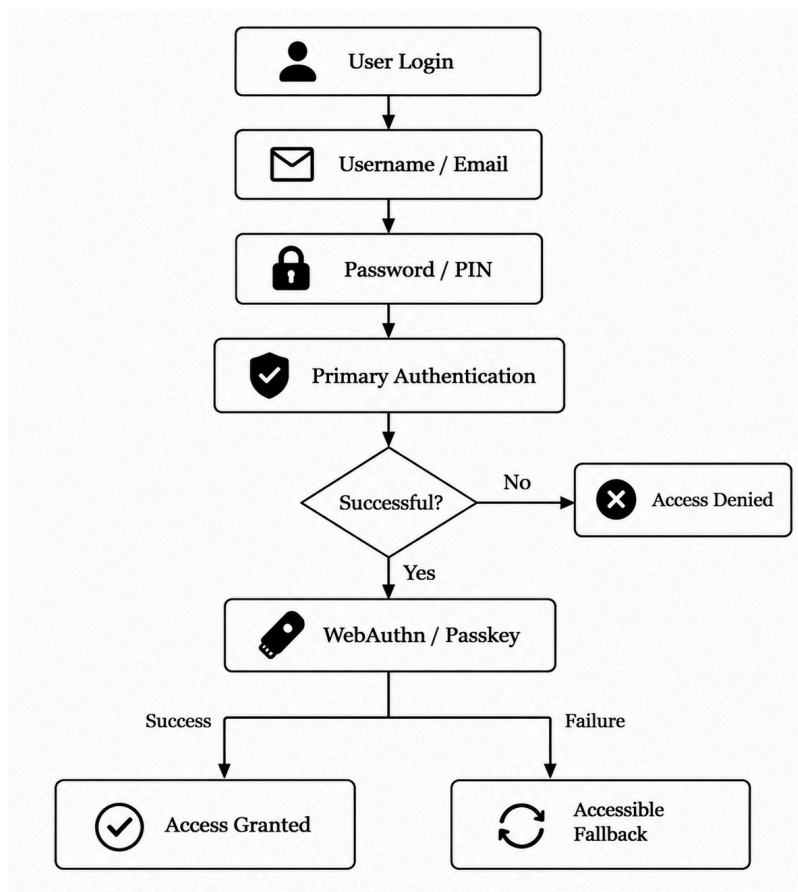
Factor Mechanism	Factor Classification	Accessibility Profile	Security Assurance	Transcription Overhead
FIDO2 / WebAuthn Security Key	Possession (Hardware)	Superior: Zero visual reading; tactile touch activation or NFC tap.	Very High (NIST AAL3, Phishing-resistant)	None (0 seconds)
Platform Biometrics (Passkeys)	Inherence + Possession	High: Direct OS integration via fingerprint or non-pose face scan.	Very High (NIST AAL3, Bound to TPM/Secure Enclave)	None (0 seconds)
Mobile One-Tap Push	Possession (Software)	High: Native OS notification action sheets read automatically by VoiceOver/TalkBack.	Moderate-High (NIST AAL2, Subject to push fatigue)	None (Single tap)

Factor Mechanism	Factor Classification	Accessibility Profile	Security Assurance	Transcription Overhead
Automated IVR Phone Call	Possession (Telephony)	Moderate: Pure audio channel; user presses physical/virtual phone keypad to approve.	Moderate (NIST AAL2, Out-of-band)	Low (Press single key)
Phonetic Audio OTP	Possession (Out-of-band)	Baseline: Spoken digits with NATO phonetic backups and playback control.	Moderate (Subject to interception/shoulder surfing)	Moderate (6-digit input)

3.3 Selected Authentication Strategy

The proposed system uses **Password/PIN + WebAuthn/Passkey** as the preferred multi-factor authentication combination.

The authentication process is designed as follows:



WebAuthn/passkeys are preferred because they provide cryptographic authentication without requiring the user to manually read and enter a short-lived verification code.

For accessibility and availability, mobile push authentication, IVR, and audio OTP can be provided as alternative verification mechanisms. These alternatives allow the system to accommodate users with different devices, assistive technologies, and authentication requirements.

Voice interaction, text-to-speech, screen-reader support, and haptic feedback are considered accessibility mechanisms rather than independent authentication factors. They help users interact with the authentication system without relying on visual information.

3.4 Authentication Decision and Fallback Strategy

The system follows a prioritized authentication strategy:

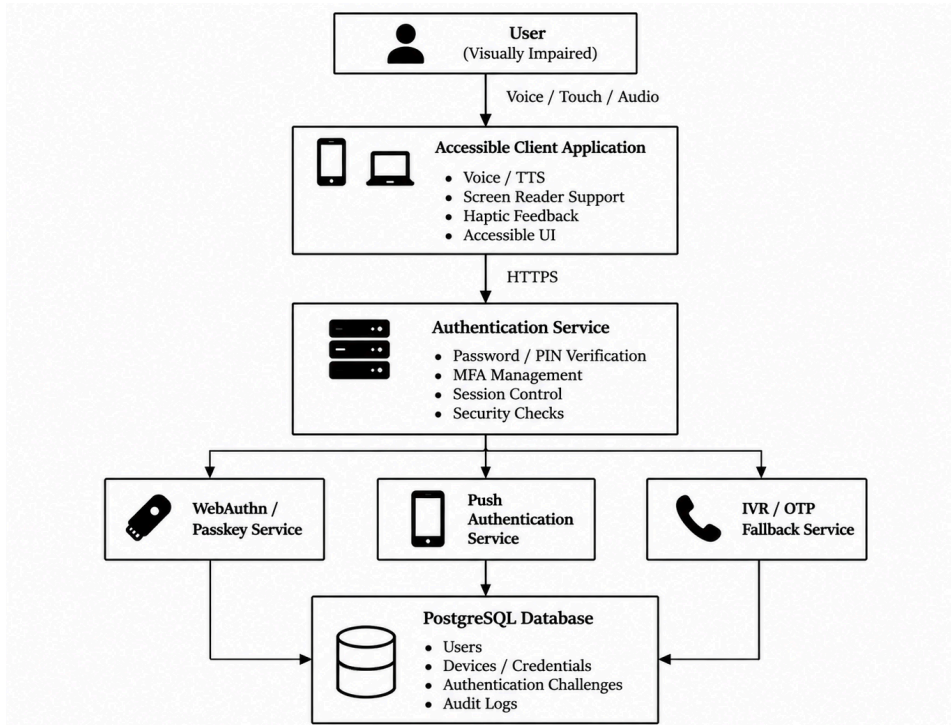
1. The user provides their username/email and password/PIN.
2. After successful primary authentication, the system requests the preferred second factor.
3. WebAuthn/passkey authentication is attempted when supported by the registered device.
4. If WebAuthn is unavailable, an accessible alternative such as mobile push authentication may be offered.
5. IVR or audio OTP can be used as additional fallback mechanisms where configured.
6. Every authentication attempt, including successful and failed fallback attempts, is recorded in the security audit log.
7. Expired, rejected, or already-consumed authentication challenges cannot be reused.

This approach maintains a balance between security, accessibility, and authentication availability.

4. Detailed Technical Architecture & Workflows

4.1 Overall System Architecture

The A-MFA system consists of an accessible client application, an authentication service, MFA services, and a secure PostgreSQL database.

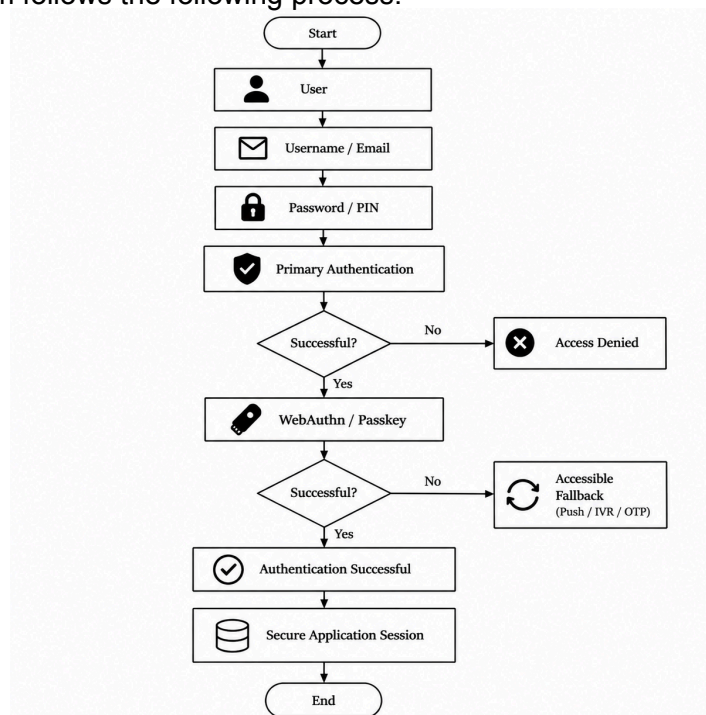


All communication between the client and authentication service is protected using HTTPS. Authentication credentials, challenges, and security logs are securely managed by the backend.

4.2 Registration and Authentication Workflow

During registration, the user creates an account and configures an authentication method. The password is securely hashed before storage, and a WebAuthn/passkey credential can be registered with the account.

During login, the system follows the following process:



The system records authentication events in the audit log and prevents expired or previously used authentication challenges from being reused.

4.3 WebAuthn / FIDO2 Authentication Workflow

WebAuthn is the preferred second-factor mechanism because it provides cryptographic authentication without requiring users to manually read and enter temporary verification codes.

1. The user enters their username/email and password/PIN through the accessible interface.
2. After successful primary authentication, the server generates a unique cryptographic challenge.
3. The client invokes the WebAuthn API.
4. The user authenticates using a security key or platform authenticator such as a passkey, fingerprint, facial recognition, or device PIN.
5. The authenticator generates a cryptographic assertion using the registered credential.
6. The server verifies the challenge, relying-party information, origin, authenticator data, and cryptographic signature.
7. If verification succeeds, the server establishes a secure authenticated session.
8. The authentication event is recorded in the audit log.

This approach minimizes visual interaction and eliminates manual OTP transcription for the preferred authentication path.

4.4 Accessible Fallback Authentication

When WebAuthn/passkey authentication is unavailable, the system provides accessible alternative mechanisms such as mobile push authentication, IVR, or audio OTP.

Mobile Push: A notification is sent to the user's registered device. The user can approve or reject the request using the device's accessible interface.

IVR: The system initiates a verification call to the user's registered phone. The user follows audio instructions and confirms the request using the telephone keypad.

Audio OTP: A short-lived OTP is delivered through an audio channel. The user can listen to the code and enter it through the accessible interface.

All fallback mechanisms use time-limited authentication challenges and record successful and failed attempts in the audit log. WebAuthn/passkey remains the preferred authentication mechanism because of its strong phishing resistance.

4.5 Technology Stack

The proposed A-MFA system uses the following technologies and standards:

Frontend / Client

- HTML5, CSS3 and JavaScript
- WAI-ARIA for accessible user interfaces
- WebAuthn Browser API for FIDO2 authentication
- Screen reader technologies such as NVDA, JAWS, VoiceOver and TalkBack

Backend

- REST-based authentication service
- HTTPS for secure client-server communication
- WebAuthn/FIDO2 server-side verification
- Authentication and session management services

Database

- PostgreSQL
- UUID-based identifiers
- Relational integrity using primary and foreign keys
- Authentication audit logging

Authentication Technologies

- WebAuthn / FIDO2
- Platform Passkeys
- Mobile Push Authentication
- IVR
- Audio OTP

Accessibility Technologies

- WAI-ARIA
- Screen Readers
- Text-to-Speech
- Keyboard-only navigation
- Haptic feedback

Security Standards and Practices

- HTTPS/TLS
- Secure password hashing
- Cryptographic authentication challenges
- STRIDE threat modeling

- WCAG 2.2 accessibility guidelines

5. Frontend Implementation & Screen Reader Accessibility Specifications

The user interface must strictly conform to WAI-ARIA authoring practices. The following architectural design patterns are mandatory for client implementations:

5.1 Single Consolidated Input & Live Region Handling

Splitting 6-digit codes across 6 individual input fields is strictly prohibited. Screen readers treat separate boxes as distinct focus points, causing disorientation during typing or paste actions. A single input field with proper auto-complete and ARIA binding must be implemented:

```
<!-- HTML Accessible Verification Pattern -->
```

```
<div class="auth-card" role="region" aria-labelledby="auth-heading">
```

```
  <h1 id="auth-heading">Two-Step Verification</h1>
```

```
  <p id="auth-instructions">
```

```
    Touch your security key, enter your verification code below, or select an alternative option.
```

```
</p>
```

```
<!-- Consolidated Single Input Field for OTP -->
```

```
<!-- Replaces the prohibited 6 split input boxes with a single accessible input -->
```

```
<div class="otp-input-group">
```

```
  <label for="otp-input">6-Digit Verification Code</label>
```

```
  <input type="text"
```

```
    id="otp-input"
```

```
    name="otp"
```

```
    inputmode="numeric"
```

```
    pattern="[0-9]*"
```

```
    maxlength="6"
    autocomplete="one-time-code"
    aria-describedby="otp-hint"
    class="single-otp-input">

<!-- Visually hidden hint specifically designed for screen readers -->
<span id="otp-hint" class="visually-hidden">Enter the 6-digit code without spaces.</span>

</div>

<!-- Primary Action: WebAuthn Trigger -->
<button id="webauthn-trigger" type="button" class="btn-primary" aria-describedby="auth-instructions">
    Activate Security Key / Biometrics
</button>

<!-- Secondary Action: IVR Fallback Option -->
<button id="ivr-trigger" type="button" class="btn-secondary">
    Call My Phone (IVR Fallback)
</button>

<!-- Managed Live Region for Asynchronous Screen Reader Announcements -->
<div id="sr-status-announcer"
    role="status"
    aria-live="polite"
    aria-atomic="true"
    class="visually-hidden">

</div>

<!-- Session Timeout Extension Mechanism -->
<div id="timeout-control" role="group" aria-label="Session duration control">
```

```

<!-- aria-live="off" prevents screen readers from repeatedly announcing every single second -->
<span id="time-remaining" aria-live="off">Time remaining: 4 minutes 30 seconds</span>

<button type="button" id="extend-timer-btn" aria-label="Add 5 minutes to verification window">
  Extend Time
</button>
</div>
</div>

```

5.2 Client-Side JavaScript Assertion Script

The client-side authentication controller manages WebAuthn interactions, user feedback loops, and error announcements:

```

// Client-Side JavaScript Assertion Script

async function executeAccessibleWebAuthn(challengePayload) {

  const announcer = document.getElementById('sr-status-announcer');

  // Announce to screen readers that the prompt is active

  announcer.textContent = "Security key prompt active. Please insert and touch your security key.";

  try {

    // Configure WebAuthn request options

    const publicKeyCredentialRequestOptions = {

      challenge: Uint8Array.from(atob(challengePayload.challenge), c => c.charCodeAt(0)),

```

```
allowCredentials: challengePayload.allowedCredentials.map(cred => ({
```

```
  id: Uint8Array.from(atob(cred.id), c => c.charCodeAt(0)),
```

```
  type: 'public-key',
```

```
  transports: ['usb', 'nfc', 'internal']
```

```
  })),
```

```
  timeout: 120000, // Generous 2-minute accessibility window as per WCAG guidelines
```

```
  userVerification: 'preferred',
```

```
  rpId: window.location.hostname
```

```
};
```

```
// Trigger the native OS WebAuthn dialog
```

```
const assertion = await navigator.credentials.get({
```

```
  publicKey: publicKeyCredentialRequestOptions
```

```
});
```

```
// Announce successful key detection before processing
```

```
announcer.textContent = "Security key detected. Verifying authentication response with server.";
```

```
// Transmit the cryptographic assertion to the server endpoint
```

```
const response = await fetch('/api/v1/auth/webauthn/verify', {
```

```
  method: 'POST',
```

```
  headers: { 'Content-Type': 'application/json' },
```

```
  body: JSON.stringify({
```

```
    id: assertion.id,
```

```
    rawId: btoa(String.fromCharCode(...new Uint8Array(assertion.rawId))),
```

```
    type: assertion.type,
```

```
    response: {
```

```
      authenticatorData: btoa(String.fromCharCode(...new Uint8Array(assertion.response.authenticatorData))),
```

```
      clientDataJSON: btoa(String.fromCharCode(...new Uint8Array(assertion.response.clientDataJSON))),
```

```
      signature: btoa(String.fromCharCode(...new Uint8Array(assertion.response.signature))),
```

```
      userHandle: assertion.response.userHandle
```

```
    ? btoa(String.fromCharCode(...new Uint8Array(assertion.response.userHandle)))
```

```
    : null
```

```
}
```

```
})
```

```
});
```

```
if (response.ok) {
```

```
    // Announce success and seamlessly redirect
```

```
    announcer.textContent = "Authentication successful. Redirecting to dashboard.";
```

```
    window.location.href = "/dashboard";
```

```
    } else {
```

```
        throw new Error("Server verification rejected.");
```

```
    }
```

```
    } catch (error) {
```

```
        // Announce failure or timeout and provide clear next steps for the user
```

```
        announcer.textContent = "Authentication attempt timed out or failed. Please select an alternative login method or try again.";
```

```
    }
```

```
}
```

6. Database Schema & Data Models

The relational model is designed using PostgreSQL, incorporating strict cryptographic storage standards, foreign key referential integrity, accessibility preference storage, and audit logs.

```
-- Enable cryptographic UUID extension
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";

-- Core User Directory
CREATE TABLE users (
    user_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL, -- Argon2id hash
    phone_e164 VARCHAR(32),
    mfa_enabled BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- User Accessibility & Factor Preferences
CREATE TABLE user_accessibility_profiles (
    profile_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(user_id) ON DELETE CASCADE,
    preferred_mfa_channel VARCHAR(32) DEFAULT 'WEBAUTHN', -- 'WEBAUTHN', 'IVR', 'AUDIO_OTP', 'PUSH'
    session_timeout_seconds INTEGER DEFAULT 300, -- Default 5 minutes for screen reader comfort
    audio_speech_rate NUMERIC(3, 2) DEFAULT 1.00, -- 0.75x to 2.0x playback speed
    phonetic_assistance_enabled BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- FIDO2 / WebAuthn Registered Authenticators
CREATE TABLE webauthn_credentials (
    credential_id BYTEA PRIMARY KEY,
    user_id UUID REFERENCES users(user_id) ON DELETE CASCADE,
    public_key BYTEA NOT NULL,
    signature_counter BIGINT NOT NULL DEFAULT 0,
    aaguid UUID,
    device_friendly_name VARCHAR(128),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    last_used_at TIMESTAMP WITH TIME ZONE
);

-- Authentication Challenges in Transit
CREATE TABLE auth_active_challenges (
    challenge_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(user_id) ON DELETE CASCADE,
```

```

    challenge_value VARCHAR(128) NOT NULL,
    channel VARCHAR(32) NOT NULL,
    expires_at TIMESTAMP WITH TIME ZONE NOT NULL,
    is_consumed BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Compliance & Security Audit Trail
CREATE TABLE auth_audit_log (
    log_id BIGSERIAL PRIMARY KEY,
    user_id UUID REFERENCES users(user_id) ON DELETE SET NULL,
    event_type VARCHAR(64) NOT NULL, -- 'LOGIN_SUCCESS', 'MFA_FAILED', 'TIMEOUT_EXTENDED'
    factor_used VARCHAR(32) NOT NULL,
    ip_address INET,
    user_agent TEXT,
    event_timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_user_credentials ON webauthn_credentials(user_id);
CREATE INDEX idx_challenges_lookup ON auth_active_challenges(user_id, is_consumed, expires_at);

```

7. Threat Modeling & Security Controls

The system implements explicit mitigations based on the STRIDE threat classification model:

Threat Vector (STRIDE)	Specific Vulnerability Context	Engineering Mitigation Architecture
Spoofing / Phishing (Man-in-the-Middle)	Adversary sets up reverse proxy (e.g., Evilginx) to intercept audio OTP or session tokens.	WebAuthn enforces cryptographic origin binding. The browser binds the cryptographic assertion directly to the verified TLS domain, preventing proxy capture.
Information Disclosure (Eavesdropping)	Screen reader speech synthesizers or audio OTPs spoken aloud in public spaces overheard by bystanders.	Push and IVR authentication require tactile button presses (e.g., "Press 1 to confirm") rather than vocalization of secret OTP codes. Headphone detection warnings provided.

Threat Vector (STRIDE)	Specific Vulnerability Context	Engineering Mitigation Architecture
Repudiation & Authenticator Cloning	Duplication of software keys or replay of previous session assertions.	Strict monotonicity validation of WebAuthn signature counters stored in the database. Any assertion with counter $\leq$ previous counter triggers immediate account quarantine.

8. Quality Assurance & Assistive Technology Testing Matrix

Prior to production deployment, the authentication flow must execute automated end-to-end accessibility testing suites and manual validation across target assistive environments:

- **Automated Accessibility Auditing:** Integrate tools like axe-core, Pa11y, or Lighthouse CI into your pipeline to catch baseline WCAG violations, missing labels, and focus-order regressions automatically on every build.
- **Manual Screen Reader Verification:** Conduct manual end-to-end test runs across the key desktop and mobile combinations (NVDA/Chrome, JAWS/Edge, VoiceOver/Safari, TalkBack/Chrome) to verify speech priority, live-region stability, and native dialog handoffs.
- **No-Mouse / Keyboard-Only Walkthroughs:** Test the complete login, failure, and timeout-extension flows using strictly standard navigation keys (Tab, Shift+Tab, Space, Enter, and shortcut keys) with a persistent, high-contrast focus indicator.
- **Peripheral & Touch Testing:** Validate physical interactions with hardware tokens (USB insertion, NFC taps on mobile devices, and biometric sensors) to ensure prompts do not trap focus or freeze screen reader audio.
- **Real-World User Testing:** Partner with visually impaired participants to complete timed scenarios across preferred assistive setups, measuring task completion rates, confusion points, and the clarity of fallback options (IVR/Audio OTP).

9. Architectural Conclusion & Roadmap

The Accessible MFA system architecture resolves the historical tension between stringent security standards and universal accessibility. By anchoring secondary authentication on hardware-backed WebAuthn protocols, eliminating manual transcription barriers, and providing

accessible out-of-band telephone fallbacks, the system delivers an uncompromised defense against modern cyber threats while ensuring seamless independence for visually impaired users.

9.1 Future Improvements

The following improvements can be considered for future versions of the A-MFA system:

1. **Expanded Passkey Support** – Support additional cross-platform passkey and authenticator configurations.
2. **Risk-Based Authentication** – Introduce contextual risk analysis using factors such as device, location and unusual login behaviour.
3. **Additional Accessible Authentication Methods** – Evaluate additional accessible authentication mechanisms while maintaining strong security requirements.
4. **Centralized Security Monitoring** – Integrate authentication audit logs with a Security Information and Event Management (SIEM) platform.
5. **High Availability and Scalability** – Introduce load balancing, service replication and database redundancy for large-scale deployment.
6. **Expanded Accessibility Testing** – Conduct larger usability studies with visually impaired users using different assistive technologies and devices.
7. **Continuous Security Testing** – Introduce automated penetration testing, dependency scanning and security regression testing into the development pipeline.

9.2 Design Limitations

The proposed architecture has several limitations. WebAuthn/passkey availability depends on browser, operating system and authenticator support. Mobile push authentication requires a registered and accessible mobile device. IVR and Audio OTP depend on telephone network availability and may provide lower security than phishing-resistant authentication mechanisms. Accessibility behaviour may also vary between screen readers, browsers and operating systems. Therefore, compatibility and real-world usability testing are required before production deployment.

10. Security and Design Considerations

The proposed A-MFA architecture incorporates security controls throughout the authentication process.

10.1 Credential Protection

User passwords/PINs are not stored in plaintext. Password credentials are securely hashed before being stored in the PostgreSQL database. Authentication data is transmitted only through protected HTTPS connections.

10.2 Challenge and Replay Protection

WebAuthn authentication uses unique, time-limited challenges. Each challenge is associated with the authentication attempt and is marked as consumed after successful verification. Expired or previously consumed challenges are rejected to prevent replay attacks.

10.3 Session Security

After successful multi-factor authentication, the system establishes a secure application session. Sessions should use secure authentication tokens or server-side sessions with appropriate expiration and protection against unauthorized reuse.

10.4 Audit Logging

Authentication events are recorded in the security audit log. The system records relevant events such as successful authentication, failed authentication, fallback usage, and session-related security events.

10.5 Accessibility and Security Balance

The architecture provides accessible authentication methods without making accessibility features independent authentication factors. Voice interaction, screen readers, text-to-speech, and haptic feedback support user interaction, while cryptographic authentication mechanisms provide the primary security protection.

11. Design Decision Summary

The main design decisions of the proposed A-MFA system are summarized below:

1. Primary Authentication – Password / PIN

Password or PIN is used as the first authentication step because it is familiar and simple for users.

2. Preferred MFA – WebAuthn / Passkey

WebAuthn/passkeys are selected as the preferred second factor because they provide strong cryptographic and phishing-resistant authentication without requiring manual OTP entry.

3. Platform Authentication – Passkey / Device Authentication

Platform authentication allows users to authenticate using supported device-based methods such as a device PIN, fingerprint, or facial recognition.

4. Fallback Authentication – Mobile Push

Mobile push authentication provides an accessible single-action verification method when WebAuthn/passkey authentication is unavailable.

5. Additional Fallback – IVR / Audio OTP

IVR and Audio OTP provide alternative authentication methods for users who cannot use the preferred authentication mechanism.

6. Database – PostgreSQL

PostgreSQL is used to securely store user information, authentication credentials, active challenges, device information, and audit records.

7. Communication – HTTPS

HTTPS is used to protect communication between the client application and authentication service.

8. Accessibility – WAI-ARIA and Assistive Technologies

WAI-ARIA, screen readers, text-to-speech, voice interaction, and haptic feedback are incorporated to support visually impaired users.

9. Security Monitoring – Audit Logging

Authentication events are recorded in audit logs to provide security monitoring and traceability.

These design decisions provide a balance between **security, accessibility, usability, and**

authentication availability while keeping WebAuthn/passkeys as the preferred authentication mechanism.

12. Conclusion

The proposed Accessible Multi-Factor Authentication (A-MFA) architecture combines secure authentication mechanisms with accessibility-focused interaction. The design uses Password/PIN as the primary authentication step and WebAuthn/passkeys as the preferred second factor, reducing the need for visually dependent OTP transcription.

The technical architecture separates the accessible client, authentication service, MFA mechanisms, and PostgreSQL data layer. The authentication workflow supports WebAuthn verification together with mobile push, IVR, and audio OTP fallback mechanisms when required.

Security controls including HTTPS communication, secure credential storage, unique authentication challenges, replay protection, session management, and audit logging are incorporated into the design. Accessibility mechanisms such as screen-reader support, text-to-speech, voice interaction, and haptic feedback allow visually impaired users to interact with the authentication process independently.

Overall, the proposed architecture provides a practical balance between **security, accessibility, and authentication availability**, forming a suitable foundation for implementation and further evaluation.

13. References

1. World Wide Web Consortium (W3C). *Web Authentication: An API for Accessing Public Key Credentials (WebAuthn)*.
2. World Wide Web Consortium (W3C). *Web Content Accessibility Guidelines (WCAG) 2.2*.
3. World Wide Web Consortium (W3C). *WAI-ARIA Authoring Practices Guide (APG)*.
4. National Institute of Standards and Technology (NIST). *Digital Identity Guidelines: Authentication and Lifecycle Management (SP 800-63B)*.
5. OWASP Foundation. *Authentication Cheat Sheet*.
6. OWASP Foundation. *WebAuthn Cheat Sheet*.
7. FIDO Alliance. *FIDO2 and WebAuthn Technical Specifications and Authentication Standards*.